**Name: Shahoo Birwadkar**
**Batch: T11**
**Roll No: 14**

# Experiment 5

Jenkins is an open-source automation server used for implementing Continuous Integration (CI) and Continuous Deployment (CD) pipelines. It helps automate the process of building, testing, and deploying applications, thereby improving development efficiency and software quality.

## 1. Overview of Jenkins

Jenkins allows developers to automatically integrate code changes into a shared repository and execute builds and tests. It supports a wide range of plugins that extend its functionality and enable integration with various tools and technologies.

Key features of Jenkins include:

- Continuous Integration and Continuous Deployment
- Plugin-based architecture
- Easy configuration through web interface
- Distributed build system
- Integration with version control systems like Git

## 2. Installation of Jenkins

Jenkins can be installed using different methods such as native installation or using Docker.

**Using Docker** The

command:

docker pull jenkins/jenkins:lts is used to download the

official Jenkins image. To run Jenkins container:

```
docker run -p 8080:8080 -p 50000:50000 jenkins/jenkins:lts
```

**Explanation:**

- Downloads Jenkins Long Term Support (LTS) image
- Runs Jenkins in a container

- Port 8080 is used to access Jenkins web interface
- Port 50000 is used for agent communication

## 3. Initial Setup of Jenkins

After installation:

- Jenkins is accessed via browser using http://localhost:8080
- An initial admin password is required, which is stored inside the container
- Unlock Jenkins using the password
- Install suggested plugins → Create an admin user

## 4. Jenkins Architecture

Jenkins follows a master-agent architecture:

- **Master (Controller)**:
  - Manages jobs and scheduling
  - Provides web interface
  - Coordinates build execution
- **Agent (Node)**:
  - Executes build tasks
  - Can be distributed across multiple systems

## 5. Integration with Tools

Jenkins integrates with multiple tools to automate workflows:

- **Version Control Systems**: Git, GitHub
- **Build Tools**: Maven, Gradle
- **Containerization**: Docker
- **Deployment Platforms**: Kubernetes, cloud services

Integration is achieved through plugins and pipeline scripts.

## 6. Jenkins Pipeline

A Jenkins pipeline is a set of automated steps defined in a script (Jenkinsfile) to execute CI/CD processes.

Types of pipelines:

- **Declarative Pipeline**: Structured and easy to read

- **Scripted Pipeline**: More flexible and uses Groovy scripting Typical stages in a pipeline:

  - Code checkout
  - Build
  - Test
  - Deploy

## 7. Benefits of Jenkins Integration

- Automates repetitive development tasks
- Detects errors early through continuous testing
- Improves collaboration among teams
- Speeds up software delivery process  Ensures consistent deployment

## Conclusion

Thus, Jenkins is a powerful tool for automating software development processes through Continuous Integration and Continuous Deployment. Its installation using Docker and integration with various tools enables efficient and reliable application development, testing, and deployment.

now theory for - create pipeline in jenkins test and deploy

Here is your **proper, structured, directly copyable theory for "Create Pipeline in Jenkins, Test and Deploy"**:

# Theory: Creating Pipeline in Jenkins for Testing and Deployment

A Jenkins Pipeline is a suite of automated processes that defines how software is built, tested, and deployed. It enables Continuous Integration (CI) and Continuous Deployment (CD) by automating each stage of the software development lifecycle.

## 1. Overview of Jenkins Pipeline

A pipeline in Jenkins is defined using a script called a **Jenkinsfile**, which contains a series of steps organized into stages. Each stage represents a phase in the lifecycle such as building, testing, and deployment.

Pipelines help in:

- Automating workflows

- Reducing manual intervention
- Ensuring consistent and repeatable processes

## 2. Types of Jenkins Pipelines

- **Declarative Pipeline**
    - Uses a structured and simplified syntax
    - Easier to read and maintain
- **Scripted Pipeline**
    - Uses Groovy scripting
    - Provides more flexibility and control

## 3. Pipeline Structure

A typical Jenkins pipeline consists of the following components:

- **Pipeline Block**: Defines the entire pipeline
- **Agent**: Specifies where the pipeline will run
- **Stages**: Logical divisions of the pipeline
- **Steps**: Individual tasks within each stage

## 4. Stages in CI/CD Pipeline

### 1. Source Stage

- Retrieves code from version control systems like Git
- Ensures latest code is used for build

### 2. Build Stage

- Compiles source code
- Installs dependencies
- Generates build artifacts

### 3. Test Stage

- Executes automated test cases
- Validates functionality of the application
- Helps in early detection of bugs

### 4. Deploy Stage

- Deploys the application to a server or container
- Can be done on local server, cloud, or Docker container → Ensures application is available for end users

## 5. Testing in Jenkins Pipeline

Testing is an essential part of the pipeline:

- Unit testing verifies individual components
- Integration testing checks interaction between modules
- Automated tests ensure code quality and reliability → Test results are displayed in Jenkins dashboard

## 6. Deployment in Jenkins Pipeline

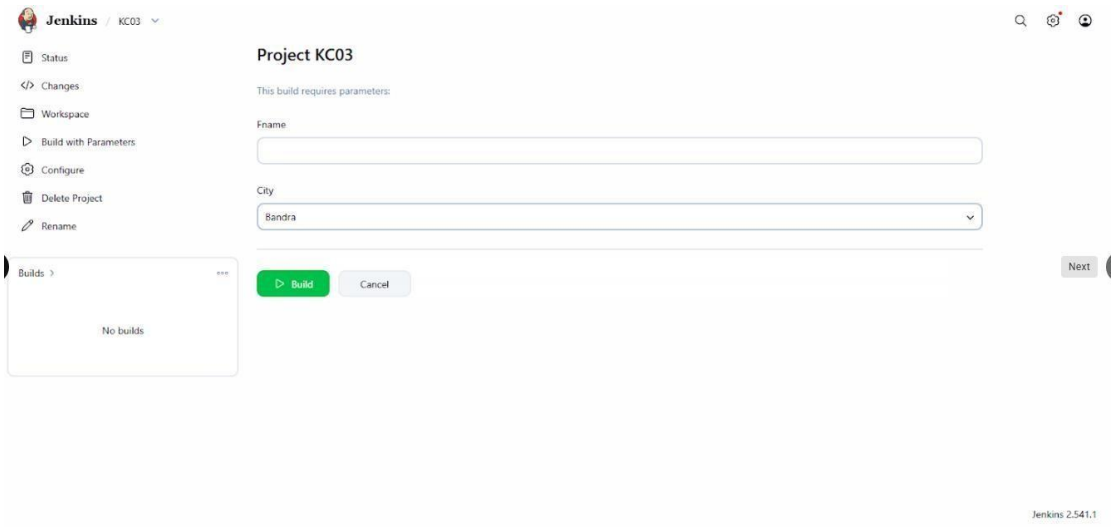
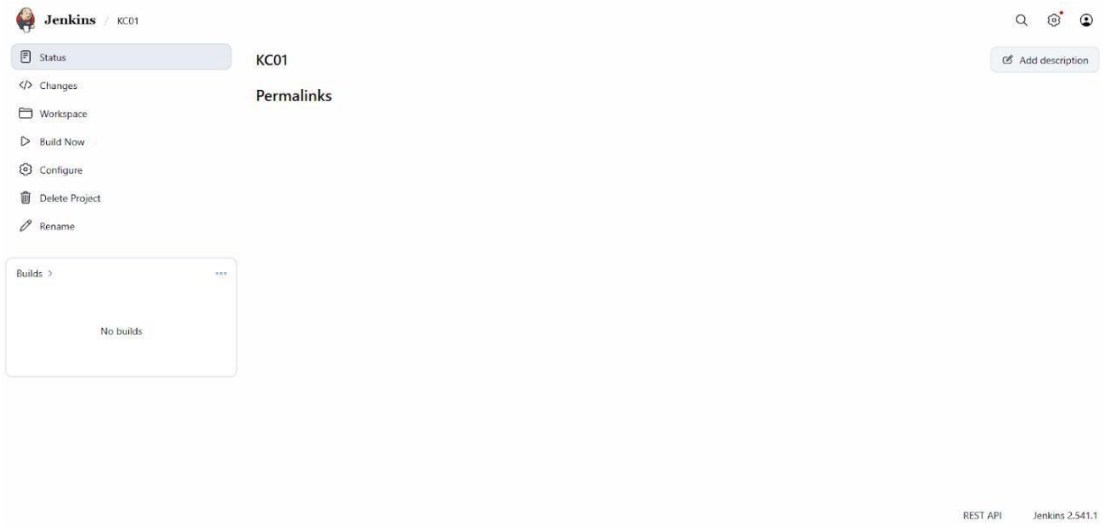
Deployment automates the release of applications:

- Can deploy to staging or production environments
- Uses tools like Docker or cloud platforms
- Ensures faster and error-free delivery

## 7. Advantages of Pipeline Automation

- Continuous feedback on code changes
- Faster development and deployment cycles
- Reduced human errors
- Improved software quality
- Easy monitoring and debugging

## Conclusion

Thus, creating a pipeline in Jenkins enables automation of building, testing, and deployment processes. It plays a crucial role in implementing CI/CD practices, ensuring faster, reliable, and efficient software delivery.

Maneesh Ala 2301003